

FORWARD-DEPLOYED ENGAGEMENT · PROOF OF CONCEPT KICKOFF

Enterprise AI Operating Model

Giving enterprise agents useful freedom without granting ambient authority — and the twelve-week plan to prove it on one real path in **[CLIENT]**'s estate.

This session has one job: leave the room with five decisions made.

Everything else is context for those five. If we run out of time, we cut architecture depth — not the decisions.

TIME	SECTION	PRIMARILY FOR
0:00	The case — why the control problem is already live	Executives · AI practice
0:12	Reference architecture — six planes, one crossing fabric	Architects · Platform
0:47	Your estate — reuse, extend, gap by discipline	All lanes
1:12	The proof of concept — scope, success, kill criteria, plan	Everyone
1:32	Decisions	Executive sponsor

Interrupt on facts, hold on opinions. If we state something about your environment that is wrong, correct us immediately — we would rather be wrong in minute 12 than build on it for a month.

LEAVING WITH

Five decisions

- 1 The **first path** and its accountable owner
- 2 The **risk tiers** and who assigns them
- 3 Reuse-first **incumbents** for identity, telemetry, gateway
- 4 The **joint team** and weekly gate cadence
- 5 The **kill criteria** — approved as written, or amended today

Six disciplines are in this room. None of them is an audience.

Each lane has a page written specifically for it. If you only have ten minutes, read slides 4, 8, 25 and 32 — that is the whole argument and the whole commitment.

DEPTH 1 · 10 MIN

Executive

What is being claimed, what it costs, what would make us stop, and what you are deciding today.

Slides 04 · 08 · 25 · 26 · 32

DEPTH 2 · 35 MIN

Architecture & platform

The six planes, the runtime path every consequential call follows, the five invariants and how each one is tested.

Slides 09 - 15 · 17

DEPTH 3 · 25 MIN

Discipline lanes

One page each for platform & DevOps, data governance, data engineering, AI practice, and security & audit — what changes, and what does not.

Slides 18 - 21

TEMPLATE

Fields in **[BRACKETS]** are discovery inputs, not claims — they are filled from the week-0 intake sheet in the appendix. Press **E** to highlight every one of them on any slide.

Your agents already have authority. Today it is ambient, unattributable, and effectively un-revocable.

Every service account, long-lived key and shared secret an agent can reach is a standing grant that no one issued, no one reviews, and no one can withdraw inside a day. The hard part was never making agents call tools. It is making every consequential action **discoverable, authorizable, constrainable, interruptible, observable and explainable.**

DISCOVER

Find capability, owner, health, version

GOVERN

Decide who may perform which action

EXECUTE

Run inside a bounded, observable path

REACT

Turn evidence into governed automation

Enterprise agent programs stall in four recognisable ways.

Not one of these is a model problem. Each is an operating-model problem wearing a model problem's clothes. Which of these four sounds most like **[CLIENT]** today?

PATTERN 01

The pilot that cannot graduate

Tell: "We have forty pilots and nothing in production."

Root cause: no control contract, so every promotion triggers a bespoke security review that never finishes.

→ Control plane

PATTERN 02

The parallel estate

Tell: the AI team has stood up its own identity, secrets and logging.

Root cause: agents treated as a new platform rather than a new class of actor inside the platform you already run.

→ Platform plane

PATTERN 03

The ontology cathedral

Tell: "We are fourteen months into the enterprise data model."

Root cause: shared semantics made a prerequisite instead of a destination, so nothing consequential ever ships.

→ Knowledge plane

PATTERN 04

The unauditable win

Tell: "It works, but Risk will not sign it off."

Root cause: "the model decided" was offered as an audit explanation, and no dashboard can repair that after the fact.

→ Telemetry fabric

The common root: all four begin by choosing products, then discover the responsibilities afterwards. This architecture inverts that order — and that inversion is the entire proposition.

**The goal is not maximum
autonomy.
It is the maximum useful
autonomy
an organization can govern
with confidence.**

Standardize the control contract. Keep every product replaceable.

We treat this as an operating-model problem before it becomes a product-selection problem. The enterprise needs stable responsibilities and contracts; the technologies underneath them should stay swappable.

Five responsibilities make up the contract: **identity, policy, capability, observability and evidence**. Fix those, and organizations remain free to choose the products that implement each one — cloud by cloud, region by region, decade by decade.

FIXED FOR TEN YEARS

Responsibilities · interfaces · evidence schema · the runtime path

CHOSEN PER CONTEXT

Policy engine · gateway · runtime · mesh · store · model

What this buys the CIO: the ability to change any single product in the stack without renegotiating the security posture of everything above it.

WHAT THE ARCHITECTURE OPTIMIZES FOR

Every action crosses an **explicit control boundary**.

Permissions can narrow through delegation but **never silently widen**.

Services are discovered **by capability and policy context** — not hard-coded endpoints alone.

Operations hold health, version, ownership, evidence **and an off switch** in one place.

Existing enterprise investments are **reused wherever they satisfy the responsibility**.

What we are asking you to believe for twelve weeks.

Stated so that it can fail. Every one of these six is instrumented, and the instrumentation is yours to keep whatever the outcome.

In twelve weeks we will place **one consequential action** behind an explicit control boundary and demonstrate six properties on it.

PROPERTY 01

Attribution

Every action traces to a human, a workload and a decision.

PROPERTY 02

Attenuation

Delegated authority narrows; it never widens.

PROPERTY 03

Revocation

Authority can be withdrawn inside a bounded, measured window.

PROPERTY 04

Containment

The blast radius of a wrong action is bounded by construction.

PROPERTY 05

Evidence

One correlated record answers an auditor without a human narrator.

PROPERTY 06

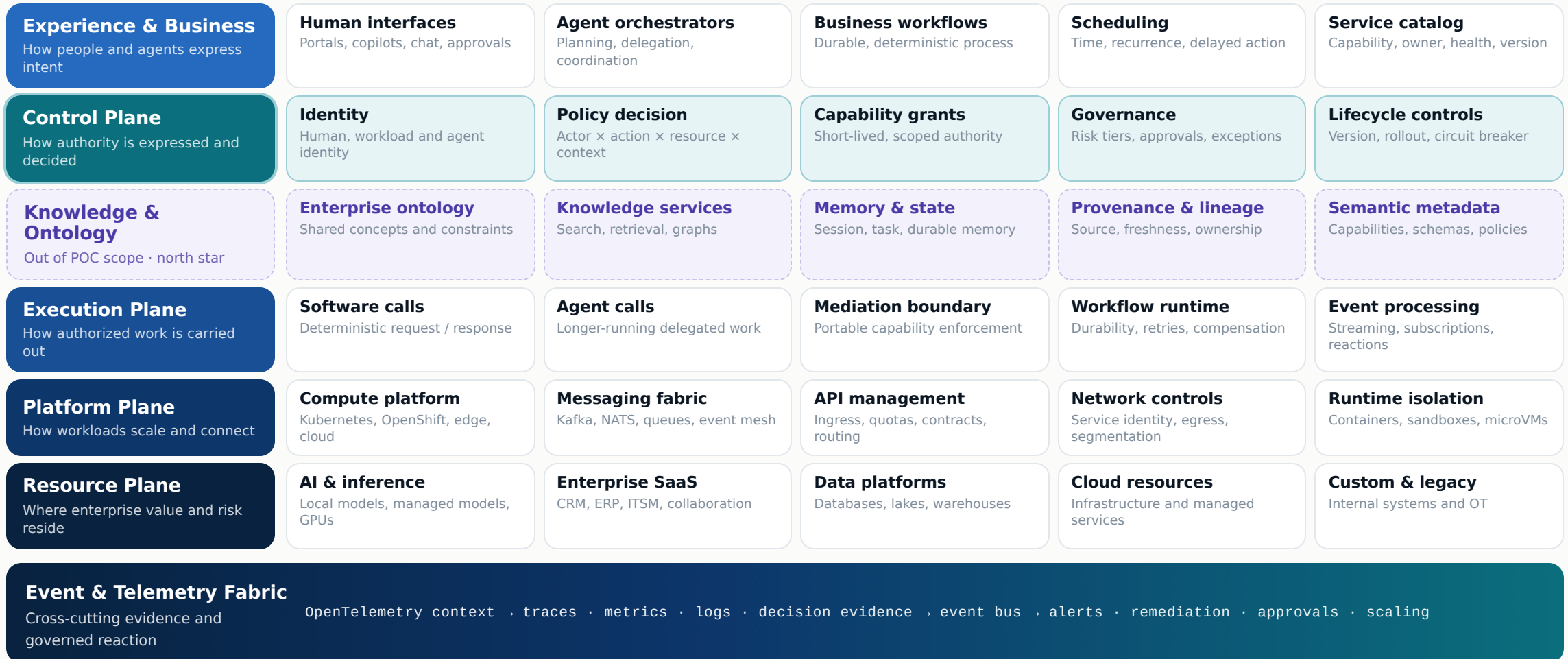
Bypass-resistance

The registered path cannot be routed around at the resource.

And the counter-commitment: if any one of the six fails to hold, the architecture is wrong for **[CLIENT]** and you should not scale it. We will say so ourselves, in week ten, in writing.

Six responsibility planes, one crossing fabric.

Responsibilities are fixed. Product selections are contextual. The control plane is the only genuinely new layer — the knowledge plane is the destination, and is deliberately out of scope for this proof of concept.



The control plane is the only genuinely new layer.

Four of its five responsibilities already exist in your estate under different names. The one that usually does not is **capability grants** — and that is where most of the engineering effort in this proof of concept goes.

RESPONSIBILITY	ANSWERS	IN ESTATE?
Identity Human, workload and agent	Who or what is acting, and on whose behalf?	● REUSE
Policy decision Actor × action × resource × context	May this actor perform this action, right now?	● EXTEND
Capability grants Short-lived, scoped, attenuable	How is that decision carried to the thing enforcing it?	● GAP
Governance Risk tiers, approvals, exceptions	Who decided this action class was acceptable, and when?	● REUSE
Lifecycle controls Version, rollout, circuit breaker	How do we stop it — in seconds, without a deploy?	● EXTEND

THE UNFAMILIAR ONE

A grant is not a role

A role describes what an actor **is**, and persists. A grant describes what an actor **may do** — bound to an audience, an action, a scope and an expiry — and is designed to die.

Four properties we require of every grant:

- **Audience-bound** — usable only at the intended resource
- **Attenuable** — a holder can narrow it, never widen it
- **Short-lived** — expiry is the default revocation mechanism
- **Verifiable offline** — routine work does not need a round trip

Design tension we will resolve in week 4, together: offline verification gives you latency; online decisions give you instant revocation. High-risk actions get the round trip. Routine actions get local verification with a short TTL. Where **[CLIENT]** draws that line is a governance decision, not an engineering one.

Every consequential call follows one explainable path.

Implementation varies by risk tier. The logical sequence does not — it stays legible to engineers, operators, security and auditors alike.

1

Express intent

A human, workflow or agent requests a *capability* — not an endpoint.

2

Establish identity

Resolve human, workload, agent type and delegated authority.

3

Discover

Find eligible services by capability, locality, health, cost, version.

4

Decide policy

Actor, action, resource, context, risk, breaker state, delegation limits.

5

Convey authority

Issue a short-lived grant bound to audience, action, scope and expiry.

6

Enforce

The mediation boundary verifies the grant and exposes only approved interfaces.

7

Execute

Run locally, at the edge or in cluster with the required isolation and limits.

8

Produce evidence

Correlate decision, grant, version, runtime, output, side effects, outcome.

```
trace_id 4f2a...c81b  on_behalf_of user: [EMPLOYEE]  actor agent:claims-triage/v3.1  workload spiffe://.../triage
capability [ACTION]  decision permit  policy tier2-writeback@v11  grant aud=claims-api scope=case:write ttl=90s
enforced_at mediation-boundary/v2.4  isolation sandboxed  side_effects 1 write · reversible  outcome ok · 214ms
```

This record — not the demonstration — is the deliverable. If an auditor can read it without a human narrating what the agent "was trying to do," the architecture is doing its job.

Shared semantics are the destination, never the prerequisite.

The control plane answers "**may this actor perform this action?**" The knowledge plane answers "**what does this request mean, what should the actor know, and which evidence supports the result?**" Keeping them distinct is what stops a vector store from quietly becoming an authorization layer.

IN POC SCOPE

Minimum viable implementation

Catalog metadata, schemas, curated retrieval sources and explicit provenance. Memory stays narrow, task-bound and disposable. Enough to prove governed invocation without making an ontology programme a dependency.

BEYOND POC

North-star implementation

A federated enterprise ontology connecting business concepts, service capabilities, policies, data products, owners and evidence. Agents request meaning and capability rather than vendor endpoints, and receive context appropriate to identity, task and risk tier.

STAGE 1

Cataloged

Named services, schemas, owners, classifications and approved sources.

STAGE 2

Semantically connected

Shared vocabulary, entity resolution, provenance and relationship-aware retrieval.

STAGE 3

Governed organizational memory

Policy-aware context, reusable knowledge products, feedback loops, measured grounding quality.

The critical separation. Ontology describes meaning. Policy determines authority. Retrieval supplies evidence. Models generate or choose actions. Combine these into one opaque "AI layer" and you create an assurance problem that no dashboard can repair afterwards.

Telemetry is what you collect. Evidence is what survives an auditor.

One invocation context is carried through policy, routing, runtime and backend. That single correlation is the difference between a dashboard and a defensible answer.

THE CONTRACT

A single **OpenTelemetry** context propagates from intent through decision, grant, enforcement, execution and outcome. Vendor-neutral by design, so the backend stays replaceable.

Emitted by the boundary — not by your service teams. Adopting the golden path means inheriting the instrumentation, not writing it.

GOVERNED REACTION

Telemetry may **trigger** automation — scaling, remediation, approvals, breaker trips. But the reaction is itself an action, and crosses the same control boundary as any other.

An auto-remediation with ambient authority is just a faster incident.

QUESTIONS THIS MAKES ANSWERABLE

Which human does this action ultimately trace back to?

Which policy version permitted it, and who approved that version?

What was the grant's scope and expiry at the moment of use?

Which service version and runtime actually executed it?

What side effects occurred, and are they reversible?

Were there attempts on this path that were **denied** — and why?

If we revoke now, how long until the last valid grant expires?

Each maps to a control on slide 21

Five invariants outlive every product. Each one has a test.

These are the rules we would use to evaluate any implementation — regardless of cloud, platform, policy engine or runtime. We run all five in front of your team in week ten.

INVARIANT	WHAT IT MEANS	HOW YOU TEST IT — AND WHAT A PASS LOOKS LIKE
1 Authority is explicit	No ambient access. Every action attributable to an identity and a decision.	Pick any action from the last 24 hours at random. Produce actor, human principal, decision, grant and expiry in under 60 seconds , without asking an engineer.
2 Delegation only narrows	Sub-agents and workflows receive subsets of authority — never amplification.	Have a sub-agent attempt an action its parent could not perform. It must fail closed , and the denial must appear in evidence with the attenuation reason.
3 Execution is bounded	Risk determines the enforcement and isolation boundary.	Call the backend directly, bypassing the mediation path. It must be refused at the resource — not merely at the gateway. Gateway-only enforcement is a fail.
4 Everything is observable	Policy, routing, runtime and business outcome share one correlated context.	One trace identifier joins intent → decision → grant → runtime → backend call → business outcome across at least three systems , with no manual stitching.
5 Technology is replaceable	Interfaces and responsibilities standardized; products chosen by context.	Swap the policy engine — or the gateway — behind the same contract. Zero service code changes , and evidence continuity is preserved across the swap.

Open invitation: your security and SRE teams design the adversarial cases for tests 2, 3 and 5. We would rather be broken by you in week ten than by an incident in month nine. If this room wants a sixth invariant, we will add it today.

What we are deliberately not doing.

Stated up front, so nobody has to discover it in week seven. Each of these is a way this class of programme reliably fails.

NOT DOING 01

A universal ontology as a prerequisite

Shared semantics grow where they materially improve discovery, policy, interoperability or assurance — bounded domain first, never enterprise-first.

NOT DOING 02

Rewriting services in WebAssembly

Portable mediation is a useful boundary pattern where the host interface matters. It is not dogma, and it is not a migration programme.

NOT DOING 03

One synchronous policy service in every request path

That is an availability incident waiting for a Tuesday. Risk tiering plus local verification with short TTLs keeps the hot path fast.

NOT DOING 04

A parallel identity, secrets or logging estate for agents

Agents are a new class of actor inside the platform you already operate — not a new platform beside it.

NOT DOING 05

A model-autonomy showcase

The demonstration is the control path and the evidence record. Impressive agent behaviour is not the deliverable and will not be scored.

NOT DOING 06

Replacing your gateway, SIEM or identity provider

If an incumbent satisfies the responsibility, it stays. We are adding a contract, not an estate.

You already own most of this. We are adding a contract, not an estate.

Our starting hypothesis, to be corrected by you in the next six minutes. The incumbent column is blank on purpose — we fill it together, live.

RESPONSIBILITY	[CLIENT] INCUMBENT	POSTURE	RESPONSIBILITY	[CLIENT] INCUMBENT	POSTURE
Enterprise identity	—	● REUSE	Durable workflow	—	● REUSE
Workload identity	—	● EXTEND	API management	—	● REUSE
Policy decision	—	● EXTEND	Messaging fabric	—	● REUSE
Authority conveyance	—	● GAP	Observability backend	—	● REUSE
Mediation boundary	—	● GAP	Telemetry contract	—	● EXTEND
Service catalog	—	● REUSE	Read the colours honestly. Seven reuse-first, three extensions, two genuine gaps. If that ratio is wrong for [CLIENT], the scope on slide 24 changes — and we would rather change it now than in week six.		
● Reuse first — incumbent satisfies the responsibility; we integrate			● Extend — incumbent is close; we add a contract or a convention		
			● Gap — no incumbent; this is where engineering effort goes		

Choose technology by responsibility, not by category.

The architecture is composable by design. Retain any layer that already satisfies its contract; replace only the gaps. We hold no commercial relationship with anything named here.

CONTROL & IDENTITY

RESPONSIBILITY	REPRESENTATIVE OPTIONS	WORKING POSTURE
Enterprise identity	Entra, Okta, Keycloak, cloud IAM, workforce federation	REUSE FIRST No parallel identity estate for agents.
Workload identity	SPIFFE/SPIRE, mesh identity, cloud workload identity	ADD WHERE NEEDED Where service accounts are too blunt.
Policy decision	OPA/Rego, Cedar, OpenFGA, IAM-native ABAC	SMALL VOCABULARY Expose business concepts, not engine syntax.
Authority conveyance	Signed tokens, caveated credentials, mTLS-bound grants	HYBRID Local for routine; online for high risk.

EXECUTION, PLATFORM & EVIDENCE

RESPONSIBILITY	REPRESENTATIVE OPTIONS	WORKING POSTURE
Workload isolation	Containers, sandboxed containers, microVMs, confidential VMs	RISK TIERED Match isolation cost to consequence.
Durable workflow	Temporal, Argo, Camunda, cloud workflow, existing BPM	DETERMINISM FIRST Process guarantees outside model improvisation.
Telemetry contract	OpenTelemetry APIs, SDKs, collectors, semantic conventions	STANDARDIZE One invocation context, end to end.
Observability backend	Grafana, Datadog, Splunk, Elastic, cloud APM, existing SIEM	REUSE FIRST No separate estate solely for agents.

Selection criteria we apply throughout: explainability of decisions · deterministic evaluation · attenuation and audience binding · offline verifiability · blast-radius containment · operational maturity in *your* hands, not ours.

You inherit a paved road, not a science project.

If adoption of the golden path requires us in the room, the product is wrong. That is a kill criterion on slide 26, not a caveat.

WHAT CHANGES

A **registration step** enters the service lifecycle: capability, owner, risk tier, schema, policy inheritance.

Deployment templates carry the **mediation sidecar or gateway plugin** and the telemetry contract by default.

A **circuit breaker** per capability — trippable without a deploy, by an on-call human.

Service pages show capability, owner, health, version, effective policy and the off switch **in one place**.

WHAT DOES NOT CHANGE

Your **cluster, mesh, gateway and CI/CD**. We meet the platform you can operate reliably.

Your **on-call rotation and runbooks** — extended, not replaced.

Your **observability backend**. We add semantic conventions to what you already run.

Your **release cadence**. Nothing here requires a platform freeze.

WHAT WE NEED FROM YOU

One **non-production namespace** and a deploy path we can iterate in — week 1.

A named **platform engineer at 1 day/week** who has authority over the deployment template.

Your **current golden-path definition**, however informal, so we extend rather than compete.

An honest answer on **p99 latency budget** for the first path. This constrains the enforcement design.

You own the golden path after week twelve. Which means you get a vote on every row here starting Monday.

A classification you cannot enforce at access time is a label, not a control.

This is the first place in the estate where the label becomes enforceable — and the last place anyone should build a shadow data programme.

DATA GOVERNANCE & RISK

Classification becomes an input to the policy decision — actor, action, resource *and data class*, evaluated at the moment of access rather than asserted in a catalog.

Purpose limitation becomes expressible. A grant carries the purpose it was issued for; using it for another purpose is a denial, and the denial is evidence.

Memory is a governed service, not a transcript dump. Narrow by default, task-bound, disposable, with retention and deletion semantics you define.

Provenance is mandatory, not optional. Source identity, timestamp and ownership travel with every retrieved fact.

We need from you: the data classes in scope for the first path, their

DATA ENGINEERING

No new pipelines in this proof of concept. Curated existing sources only. If it needs an ingestion project, it is out of scope by definition.

Retrieval is evidence, never authorization. Access filtering happens in the control plane; the retrieval layer is never asked to be a security boundary.

Data products gain a capability contract — what it answers, freshness, owner, classification, policy context — reusing your catalog, not replacing it.

Grounding quality is measured, not asserted. Citation coverage and source freshness are instrumented from week 6.

We need from you: read access to the catalog, one named data engineer at **0.5 day/week**, and the freshness SLA of the sources the first path

On the ontology programme: if **[CLIENT]** has one in flight, we are neither competing with it nor waiting for it. Everything this POC emits — capability semantics, provenance, ownership, policy context — is a structured input that programme can consume later. Slide 12 is where that boundary is drawn.

This exists so you can say yes more often.

Today every consequential capability triggers a bespoke review. Afterwards, tier 1 and tier 2 ship on a paved road and only tier 3 needs a conversation. That is the whole commercial argument for the control plane.

THE UNLOCK

A capability that fits an approved risk tier **inherits its policy** rather than negotiating one.

Promotion from pilot to production becomes a **checklist**, not a committee.

Agent-to-agent delegation becomes **expressible and safe** — the attenuation property is what makes multi-agent designs governable at all.

Failed and denied actions become **training signal**, because denials are evidence rather than silent errors.

NOT CONSTRAINED

Which models you use, hosted or managed, and where inference runs.

How agents are prompted, planned or orchestrated. The boundary sits between the model's output and the world's state — not inside the model.

Your evaluation stack and experimentation cadence.

Framework choice. The contract is at the invocation boundary, so frameworks stay replaceable too.

WHAT GENUINELY GETS HARDER

An agent can no longer hold a **long-lived credential** "just to make the demo work." That pattern is closed deliberately.

Capabilities need a **declared scope** before first use. Roughly a day of work per capability, once.

"The model decided" stops being an **acceptable explanation** for a consequential action.

We need from you: the capability inventory for the first path, and one named practitioner at **1 day/week**.

We do not propose a new control framework. We emit evidence the old ones accept.

Architecture evidence is translated into the frameworks [CLIENT] is already assessed against. Which of these bind you? Mark them now and we will scope the week-9 crosswalk accordingly.

ARCHITECTURE EVIDENCE	NIST AI RMF	ZERO TRUST · SP 800-207	ISO/IEC 42001	SOC 2 · EU AI ACT
Attribution record Actor, human principal, decision	MAP · MANAGE	Policy decision point logging	Operational planning & control	CC6 logical access · Art. 12 record-keeping
Grant scope & expiry Audience, action, TTL, attenuation	GOVERN · MANAGE	Per-session least privilege	Resource & access control	CC6 logical access
Denial evidence Refused attempts and reasons	MEASURE	Continuous verification	Performance evaluation	CC7 system operations
Correlated invocation trace Intent → outcome, ≥3 systems	MEASURE · MANAGE	Monitoring & diagnostics	Monitoring, measurement, analysis	CC7 · Art. 12 logging
Circuit breaker & revocation Time-to-stop, measured	MANAGE	Dynamic policy enforcement	Incident & nonconformity	CC7 · Art. 14 human oversight
Risk tier assignment Who decided, when, on what basis	GOVERN · MAP	Asset & resource classification	AI risk & impact assessment	Art. 9 risk management

Stated plainly, so nobody leaves with the wrong impression: a twelve-week proof of concept produces *evidence samples* mapped to these controls. It does not produce a certified control, an audit pass, or a compliance assertion. What it produces is a defensible case that the evidence a full implementation would generate is the evidence your assessors need.

CROSSWALK DOCUMENT DELIVERED WEEK 9 · REVIEWED WITH AUDIT BEFORE READOUT

One path, end to end, with the evidence to prove it — and the criteria to stop it.

01

path — one consequential action, taken end to end through the control boundary

06

properties — attribution, attenuation, revocation, containment, evidence, bypass-resistance

12

weeks — four phases, four gates, a written go / no-go at each one

Scope discipline is not modesty. It is the only reason a twelve-week commitment is credible at all.

The right first path is consequential, reversible, owned, instrumented — and boring.

Five criteria. We will score candidates against them live, right now, and whatever wins becomes **[FIRST PATH]** for the remaining eleven slides.

1

Consequential
It writes, moves money, changes state or touches a customer. If nothing can go wrong, governing it proves nothing.

2

Reversible
A wrong action can be undone inside a working day, by a documented procedure that already exists.

3

Singly owned
One team can say yes without convening four others. Multi-owner paths are phase-two work.

4

Instrumented
It already emits enough telemetry that we can measure a before and an after, rather than inventing a baseline.

5

Boring
Nobody is politically defending it. Contested paths turn a twelve-week engagement into a twelve-week negotiation.

LIVE SCORING — 0 (POOR) TO 3 (STRONG)

CANDIDATE PATH	OWNER	CONSEQUENTIAL	REVERSIBLE	OWNED	INSTRUMENTED	BORING	TOTAL
Candidate A — [NAME]	—	—	—	—	—	—	—
Candidate B — [NAME]	—	—	—	—	—	—	—
Candidate C — [NAME]	—	—	—	—	—	—	—

The trap to avoid: choosing your most strategically important workflow. That is the second path. Its owner will thank you for not being the one we learned on.

In scope and out of scope, stated explicitly.

Is there anything on the right that you believed was on the left? This is the cheapest possible moment to find out.

● IN SCOPE · 12 WEEKS

One consequential action on **[FIRST PATH]**, end to end through the control boundary

Identity integration with the incumbent provider — human, workload and agent

A policy decision point with a **small, business-legible vocabulary** and three risk tiers

Capability grants: audience-bound, attenuable, short-lived, offline-verifiable

One mediation boundary enforcing grants in front of one real backend

Telemetry contract and correlated evidence record across at least three systems

Circuit breaker with a measured time-to-stop

Golden-path templates, service page, runbook and the five invariant tests

Control crosswalk with sample evidence, reviewed with your audit team

● OUT OF SCOPE · PHASE TWO OR LATER

A second path, or generalisation across service families

Enterprise ontology, knowledge graph or semantic federation

New data pipelines, ingestion projects or retrieval infrastructure

Production traffic, production data, or any irreversible action

Replacing the identity provider, gateway, SIEM or observability backend

Model selection, fine-tuning, evaluation harnesses or prompt engineering

Multi-region, disaster recovery or capacity engineering

Certified controls, audit sign-off or a compliance assertion

Organisational change management beyond the one owning team

How we handle a scope request mid-engagement: anything added must displace something of equal size, and the trade is written into the weekly gate note with both names on it. We will not silently absorb scope and we will not silently drop it.

Seven measurements. Every threshold set in week one and frozen in week two.

Targets are bracketed because we will not invent numbers about an estate we have not measured yet. Nobody moves a goalpost in week nine.

	CRITERION	THE MEASUREMENT	TARGET	INSTRUMENT · WHO READS IT
1	Attribution latency	Time to produce actor, human principal, decision, grant and expiry for a randomly selected action	< [60] SEC	Evidence query · your SRE, unaided
2	Enforcement overhead	Added p99 latency on the golden path versus the measured pre-POC baseline	< [X] MS	Your APM · platform engineering
3	Time to stop	From breaker trip to last valid grant expiring, measured under sustained load	< [X] SEC	Live exercise · SRE and security
4	Attenuation integrity	Adversarial cases attempting privilege amplification through delegation, designed by your team	100% FAIL CLOSED	Test suite · your security team
5	Bypass resistance	Direct backend calls that route around the mediation boundary	100% REFUSED AT RESOURCE	Test suite · your security team
6	Evidence completeness	Proportion of actions whose full record reconstructs across ≥ 3 systems with no manual stitching	> [99] %	Trace analysis · audit reviews
7	Unassisted adoption	A second team onboards a capability to the golden path using documentation alone, with us out of the room	< [1] DAY	Observed exercise · week 11

Criterion 7 is the one that predicts whether this survives us. The other six prove the architecture works. This one proves it works without its authors in the room — which is the only version worth scaling.

The conditions under which we will tell you to stop.

A kill criterion is not pessimism — it is what makes a success criterion mean something. Each is checked at a named gate. We would like a sixth from this room.

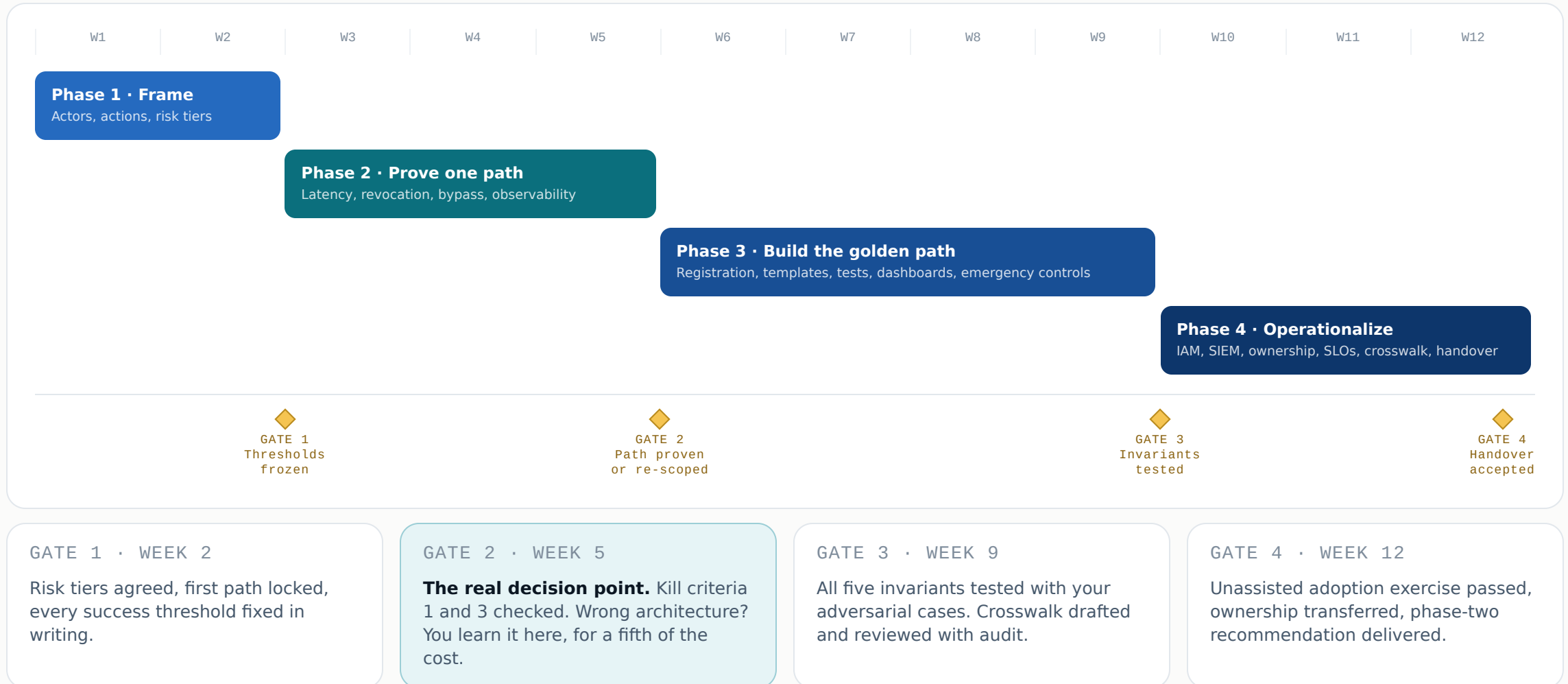
	IF THIS IS TRUE...	...THEN THIS IS WHAT IT TELLS US	CHECKED AT
1	Policy decision adds more than [X] ms p99 to the golden path and cannot be reduced by caching, attenuation or tiering	The enforcement model is wrong for this estate. A control nobody can afford to keep on is not a control.	GATE 2
2	Revocation consistently takes longer than the grant TTL under sustained load	The conveyance model is wrong . If you cannot stop it faster than it expires, the off switch is decorative.	GATE 3
3	The backend can be reached directly, and closing that route requires rewriting it	The boundary is in the wrong place for this system class — and possibly for most of your estate.	GATE 2
4	Evidence cannot be joined into one trace across three or more systems without manual stitching	The telemetry contract is wrong , and every downstream assurance claim rests on sand.	GATE 3
5	The owning team will not adopt the golden path without us physically in the room	The product is wrong . This is the failure mode that kills programmes six months after the consultants leave.	GATE 4
6	[YOUR SIXTH — added today]	Proposed by this room. We will write it in before we leave.	—

How a kill decision actually works. Any criterion tripping produces a written finding within 48 hours, a joint review inside a week, and one of three outcomes: **remediate** with a named change and a new date, **re-scope** to a path where the architecture fits, or **stop** — with the evidence, the instrumentation and the crosswalk handed over regardless. You keep the artefacts in all three cases.

Twelve weeks, four phases, four written gates.

Each gate is a one-page go / no-go, circulated 24 hours ahead so nobody reads it for the first time in the room. Starting

[START DATE].



One team with two employers.

This work succeeds when different forms of rigour challenge each other. We need a name against every row on the right before we leave this room — or a date by which we will have one.

WE BRING

Capability-security thinker

Sees ambient authority, confused deputies, replay and privilege amplification before they become implementation details.

Distributed-systems pragmatist

Assumes delay, duplication, partition, stale state and partial failure — and makes degraded behaviour explicit.

Platform product engineer

Turns control complexity into paved roads, service pages, templates and safe defaults teams willingly adopt.

Runtime performance engineer

Measures cold start, steady-state overhead, concurrency and host-call cost rather than debating by instinct.

YOU NAME — TODAY

ROLE	COMMITMENT	NAME
Executive sponsor Owns gate decisions and scope trades	Gates only	—
First-path owner Can say yes without convening four teams	0.5 day/wk	—
Platform engineer Authority over the deployment template	1 day/wk	—
Security / IAM engineer Designs the adversarial cases	1 day/wk	—
Data engineer Catalog, sources, freshness	0.5 day/wk	—
AI practitioner Capability inventory and agent design	1 day/wk	—
SRE / assurance challenger Exercises breakers, revocation, recovery	0.5 day/wk	—

Unnamed roles are the single most reliable predictor of a slipped gate.
Not underestimated effort — unnamed people.

What we need from you, and when.

If access lands in week three instead of week one, gate 2 moves to week seven. That is not a complaint — it is arithmetic, and it is the most common way this plan slips.

ACCESS

Non-production namespace and deploy path	W1
Identity provider tenant — non-prod, with admin	W1
Read access to the service catalog / CMDB	W1
Observability backend — write and query	W2
First-path backend, non-prod instance	W2
Gateway / mesh configuration in non-prod	W3

Honest question: how long does a namespace and an IdP tenant genuinely take here — not the SLA, the reality? If it is six weeks, it starts before the contract is signed.

DECISIONS

First path confirmed, with accountable owner	Today
Three risk tiers defined, and who assigns them	W1
Reuse-first incumbents confirmed	W1
Data classes in scope — and the one that is not	W1
Success thresholds fixed in writing	W2
Policy vocabulary owner named	W2
Adversarial test cases agreed with security	W4

PEOPLE & RHYTHM

Seven named roles — slide 28	Today
Weekly 45-minute working session, fixed slot	W1
Gate reviews — 60 minutes, sponsor present	W2/5/9/12
Shared channel and a single written decision log	W1
Access to your change-freeze calendar	W1

One question we need answered today: what falls inside this twelve-week window on your side — change freezes, audits, quarter close, holidays? Freezes have derailed more of these than any technical risk on the next slide.

What could go wrong, and what we do about it.

Note the pattern: almost every mitigation is early detection rather than clever engineering. These are cheap in week two and expensive in week nine.

RISK	MITIGATION	OWNER
1 Environment access slips beyond week 2	Access requests raised on day 1, tracked in the decision log, escalated at the weekly session. Gate 1 will not be declared complete without it.	Sponsor
2 The first path turns out to be contested or multi-owner	Score three candidates today, not one. A named fallback candidate carries no additional cost if we switch before week 3.	Joint
3 The backend cannot enforce at the resource without a rewrite	Tested in week 3, not week 9. If true, this is kill criterion 3 and we re-scope to a path where the boundary fits.	Us
4 Policy latency exceeds the tolerable budget on the hot path	Latency budget agreed in week 1 before design. Tiering, caching and local verification are designed in from the start, not retrofitted.	Us
5 A change freeze falls inside the window	Freeze calendar reviewed in week 1; phase boundaries shifted around it rather than compressed into it.	Platform
6 Key people are pulled onto an incident for two weeks	Named deputy for each of the seven roles. Written decision log means context survives an absence.	Sponsor
7 Scope widens after an encouraging demo	Every addition displaces something of equal size, recorded in the gate note with both names on it.	Joint

ASSUMPTIONS WE ARE MAKING

Is any one of these false? Three of the four change the plan materially.

→ A non-production environment exists that **resembles production** closely enough for latency numbers to mean something.

→ There is an **incumbent identity provider** we can federate with, rather than a greenfield decision.

→ The first-path backend can be **configured to refuse un-mediated calls** — even if that configuration does not exist yet.

→ **[CLIENT]** wants a **capability it operates**, not a demonstration it commissions.

The fourth is the one that matters most. If what you want is a demonstration, we should design a very different twelve weeks — and we would say so.

Every item is a noun you can point at.

No workstreams, no alignment, no enablement. If it cannot be opened, run or read, it is not on this slide.

DAY 1 · END OF WEEK 1

A written **autonomy frame** — actors, actions, data classes, reversibility, consequence

Three **risk tiers**, defined, with an assignment owner

The **first path** named, with an accountable owner

Seven **named people** and a fixed weekly slot

A **decision log** with entries in it

Access requests **raised**, with dates

DAY 30 · END OF WEEK 4

One real service **running behind the control boundary** in non-production

Grants being **issued, verified and expiring** under real invocation

A **correlated evidence record** you can query yourself

Measured **enforcement overhead** against a real baseline

A working **circuit breaker** with a measured time-to-stop

A frank week-5 gate paper — **including whether to stop**

DAY 90 · END OF WEEK 12

A **golden path**: registration, policy inheritance, templates, tests, dashboards, emergency controls

Five **invariant tests** in your CI, run against your adversarial cases

A **service page** showing capability, owner, health, version, effective policy and the off switch

A **control crosswalk** with sample evidence, reviewed by your audit team

A **runbook** and a second team that onboarded without us

A **phase-two recommendation** — including the honest case for not doing one

All of it is yours whether or not you continue with us. A capability you cannot operate without its authors is not a capability.

Five decisions, and we start Monday.

Taken one at a time. Where we cannot decide today, we need a name and a date rather than a maybe — an undecided item with no owner becomes a slipped gate in week three.

DECISION	WHAT WE NEED TO HEAR	OWNER	STATUS
1 The first path	A named path and an accountable owner who can say yes without convening four other teams. Plus a named fallback candidate.	—	OPEN
2 Risk tiers	Three tiers, roughly defined, and who has the authority to assign a capability to one. We will sharpen the definitions in week 1.	—	OPEN
3 Reuse-first incumbents	Confirmation of the identity provider, observability backend and API gateway we integrate with rather than replace.	—	OPEN
4 The joint team and cadence	Seven names from slide 28 — or a date by which we will have them — and a fixed weekly slot starting this week.	—	OPEN
5 The kill criteria	Approved as written, tightened, or amended today — including whatever sixth criterion this room wants to add.	—	OPEN

WITHIN 24 HOURS

Written confirmation of every decision above, the access requests raised with dates, and the first weekly session in calendars. Nothing agreed in this room survives on memory alone.

WITHIN 5 WORKING DAYS

The week-1 autonomy-framing workshop — half a day, the seven named people, and the estate map on slide 16 completed properly rather than live-edited.

**Formal enough to preserve
the security properties.
Pragmatic enough to ship one
useful
service before building a cathedral.**

Week-0 intake — the fields this deck is parameterised on.

Every bracketed field in the preceding 33 slides resolves from this sheet. Press **E** on any slide to highlight them.

ENGAGEMENT			FIRST PATH & THRESHOLDS			ESTATE & PEOPLE		
[CLIENT]		—	[FIRST PATH]		—	Identity provider		—
[DATE]	· kickoff	—	[ACTION]	· the capability	—	Observability backend		—
[START DATE]	· week 1	—	Fallback candidate path		—	API gateway / mesh		—
[ENGAGEMENT ID]		—	Attribution latency target		—	Compute platform		—
[DOMAIN]	· business area	—	Enforcement overhead budget · p99		—	Service catalog / CMDB		—
Binding regulatory frameworks		—	Time-to-stop target		—	Workflow engine		—
Change freezes in window		—	Evidence completeness target		—	Data classes in scope · and out		—
			Unassisted adoption target		—	Seven named roles · slide 28		—
Where this arrives filled in before the session, the kickoff runs thirty minutes shorter and slide 16 becomes a review rather than a live exercise. That is the single highest-leverage piece of preparation available.								

Six terms this deck uses precisely.

Each is commonly used loosely. Where a definitional argument starts, it usually ends here.

Ambient authority

Permission an actor holds by virtue of *where it is running* rather than by an explicit, attributable grant. A long-lived service-account key on a pod is ambient authority. It is the default state of most estates and the thing this architecture exists to remove.

Capability grant

A short-lived, audience-bound, attenuable credential conveying a *specific* authorization decision to the point that enforces it. Distinct from a role, which describes what an actor is and persists indefinitely.

Attenuation

The property that a holder of authority can pass on a *narrower* version of it and never a wider one. This is what makes multi-agent delegation governable rather than merely observable.

Mediation boundary

The enforcement point that verifies a grant and exposes only the approved interfaces of a resource. May be a sidecar, a gateway plugin, a portable runtime or a native component — the pattern is fixed, the implementation is not.

Golden path

The supported, templated, instrumented route by which a team registers and operates a capability. Its quality is measured by adoption without its authors present — success criterion 7.

Evidence

Telemetry correlated well enough to answer a question under adversarial scrutiny. Distinct from logs, which are what you have, and dashboards, which are what you look at.